

TRES EN RAYA

Un proyecto profesional con Jira, GitHub y Jenkins

Hecho por:	Antonio Valero Maldonado
Qué es:	Juego interactivo Tres en Raya (Vue.js 3)
Código:	github.com/toniivm/tres-en-raya
Juega aquí:	https://tres-en-raya-edf59.web.app
Fecha:	12 de May de 2026

¿De qué va esto?

Cuando empecé este proyecto pensaba que era "simplemente hacer un juego en Vue". Pero rápidamente me di cuenta de que era mucho más: aprender a trabajar como lo hacen en las empresas de verdad.

La idea era simple: hacer un Tres en Raya funcional. Pero el reto de verdad era hacerlo de forma "profesional": usando Jira para organizar el trabajo, GitHub para controlar versiones, Jenkins para automatizar el despliegue... todo eso junto.

Honestamente, al principio me abrumó un poco ver todos esos nombres y herramientas. Pero conforme avanzaba el proyecto, empecé a entender por qué cada una existe y para qué sirve. Este documento es mi reflejo de todo lo que aprendí en el camino.

Lo que hicimos

La aplicación

Un juego Tres en Raya que funciona completamente en el navegador. Te deja poner los nombres de los dos jugadores, jugar partidas infinitas y te va llevando la puntuación de victorias durante la sesión.

Suena simple, pero detrás hay lógica de programación, tests unitarios, validación de código, y un pipeline automático que se asegura de que cuando subo cambios no rompo nada.

Lo importante (más allá del juego)

- No trabajar directamente en la rama main (básico en equipos reales)
- Cada tarea en su propia rama feature/TER-X
- Commits con un formato específico (SCRUM-3: descripción del cambio)
- Pull Requests donde otros revisan antes de mergear
- Tests unitarios para asegurar que funciona
- Un pipeline que ejecuta esos tests automáticamente
- Si todo pasa, deploy automático a la web
- Si algo falla, el despliegue se cancela (safety net)

Herramientas (y por qué las usamos)

Jira - El gestor de tareas

Básicamente un tablero donde pones las cosas que tienes que hacer. Nosotros creamos 7 tareas (una por cada funcionalidad) y las fuimos moviendo de "Por Hacer" → "En Curso" → "In Review" → "Hecho" conforme avanzaban. Al final, las 7 estaban en "Hecho". Simple pero efectivo.

GitHub - Control de versiones

Donde guardamos todo el código. Pero no es simplemente "guardar archivos" como en Dropbox. GitHub mantiene el historial completo: quién cambió qué, cuándo, y por qué (a través del mensaje del commit). Además, las ramas te permiten trabajar en varias cosas a la vez sin pisar al otro.

Jenkins - El robot que verifica

Cada vez que hago push a GitHub, Jenkins automáticamente se pone a trabajar: descarga el código, instala las dependencias, ejecuta los tests, valida el código con ESLint, genera el build, y si todo está bien, despliega a Firebase. Si algo falla, el deploy no sube y me avisa. Es como tener un colega vigilando que no haga tonterías.

Vue.js 3 - El framework

Es con lo que escribimos la aplicación. Me gustó bastante porque es bastante legible y Composition API (que es lo que usamos) se siente más moderna que Options API.

Vitest - Tests

Para comprobar que la lógica del juego funciona en todos los casos posibles. Escribí 14 tests que cubren las 8 maneras de ganar, empates, cambios de turno... Si paso los tests, sé que la lógica es correcta.

Jira: Organizando el trabajo

Creé un proyecto Scrum llamado "tres en raya" con 7 historias de usuario. Cada una representaba una funcionalidad completa del juego. Al principio me pareció excesivo ("es un juego, ¿para qué tantas tareas?"), pero luego vi que tenía sentido: cada tarea era independiente y se podía hacer una rama de Git para ella.

Las 7 tareas que creé

SCRUM-1	Setup inicial	5	Configurar el proyecto Vue + Vite
SCRUM-2	Tablero 3x3	5	Los componentes GameBoard y GameCell
SCRUM-3	Lógica de victoria	3	Detectar ganador y empate
SCRUM-4	Marcador y nombres	2	Input de nombres y contador de victorias
SCRUM-5	Animaciones	3	Estilos visuales y efecto pulse en ganador
SCRUM-6	Tests	5	14 tests unitarios con Vitest
SCRUM-7	CI/CD	5	Jenkins pipeline + Firebase deploy

Al final, las 7 estaban en "Hecho". Total: 28 story points entregados. ✓

GitHub: Flujo profesional

Cada tarea de Jira tenía su propia rama en GitHub. Por ejemplo, para SCRUM-2 creé feature/TER-2-tablero. Trabajé ahí, hice commits locales, y cuando terminé la funcionalidad, abrí una Pull Request.

En una empresa real, alguien reviería mi código antes de mergear. Aquí, GitHub Copilot hizo esa revisión automáticamente. No es lo mismo que un humano, pero daba feedback útil igual.

El flujo que seguí cada vez

- 1. Crear rama desde Jira (me lleva a GitHub automáticamente)
- 2. Trabajar en local, hacer commits con formato SCRUM-X: descripción
- 3. Hacer push a GitHub
- 4. Abrir PR con descripción de lo que cambié
- 5. Jenkins automáticamente valida (lint, tests, build)
- 6. Si todo pasa, Jenkins pone un check ✓
- 7. Yo mergeo la PR a main
- 8. Jenkins automáticamente despliega a Firebase

Pull Requests (5 en total)

Todas mergeadas a main. Cada una incluía descripción, tests pasando, y la tarea de Jira que cierra. Lo que más me gustó fue ver el check "Verified" en los commits, que significa "Jenkins lo validó".

Jenkins: La automatización

Aquí es donde pasó la "magia". Configuré un pipeline que se ejecuta automáticamente cada vez que hago push a main. El pipeline hace 6 cosas seguidas, y si alguna falla, para ahí y me avisa.

Las 6 etapas

Checkout	Descarga el código más reciente de GitHub
Install	npm ci para instalar dependencias limpias
Lint	ESLint revisa que el código sea limpio (0 errores)
Test	Vitest ejecuta los 14 tests (todos pasan)
Build	Vite genera el bundle optimizado para producción
Deploy	Firebase CLI sube el bundle a Firebase Hosting

El Build #5 (cuando salió verde)

Build #5: SUCCESS en 55 segundos. Las 6 etapas completadas sin errores. Honestamente, fue uno de los momentos más satisfactorios del proyecto. Después de los fallos de los builds 1-4 (errores de configuración, paths equivocadas, etc.), verlo pasar fue increíble.

Problemas que encontré (y cómo los arreglé)

No todo fue fluido. Aquí están los obstáculos principales y cómo los resolví.

1. Jenkins no encontraba Node.js

Problema: El pipeline fallaba porque Node.js no estaba en el PATH de Jenkins. Intenté instalar el plugin de NodeJS, pero eso llevaba a más complicaciones.

Cómo lo arreglé: Al final, en lugar de usar un plugin, simplemente añadí la ruta completa de Node.js al PATH de Jenkins en el Jenkinsfile: 'C:\\Program Files\\nodejs'. Funcionó inmediatamente.

Lección: A veces la solución más simple es la correcta. No siempre hay que instalar herramientas si puedes usar lo que ya tienes.

2. Jenkinsfile en la carpeta equivocada

Problema: Había puesto el Jenkinsfile en la carpeta raíz (3Eva/), pero Jenkins esperaba encontrarlo en la raíz del repositorio (tres-en-rama/).

Cómo lo arreglé: Simplemente lo moví de carpeta. Un error tonto, pero que causó horas de confusión porque pensaba que era un problema de configuración.

Lección: La mayoría de los errores en DevOps son cosas simples de path y configuración. Leer los logs completos es crucial.

3. Firebase project ID incorrecto

Problema: El archivo .firebaserc tenía un ID de ejemplo ("your-firebase-project-id") en lugar del verdadero. El deploy fallaba sin explicación clara.

Cómo lo arreglé: Fui a la consola de Firebase, copié el ID real (tres-en-rama-edf59), y lo pegué en .firebaserc.

Lección: Siempre revisar las configuraciones antes de intentar usarlas. Los ejemplos en archivos de template están ahí por algo.

4. ESLint warnings rompiéndolo todo

Problema: ESLint encontraba warnings menores (indentación, comillas simples vs dobles) y Jenkins los trataba como errores (exit code 2 = fallo).

Cómo lo arreglé: Ejecuté 'npx eslint --fix' localmente, que auto-corrigió todo automáticamente. Luego committé esos cambios.

Lección: Usar --fix en herramientas de linting es mucho más eficiente que arreglar manualmente. Tiempo es oro.

5. Git rechazando el push

Problema: Después de mergear PRs, intenté hacer push y GitHub me rechazaba porque la rama remota tenía commits que yo no tenía.

Cómo lo arreglé: Hice 'git pull --rebase origin main' para sincronizar antes de push. Esto es más limpio que un merge porque no crea commits extra.

Lección: En equipos, siempre pullear antes de pushear. Es una práctica básica que evita conflictos innecesarios.

6. Tests con jsdom dando errores raros

Problema: Vitest con jsdom daba errores cuando intentaba testear cosas que tocaban el DOM. No sabía si era configuración o mi código.

Cómo lo arreglé: En lugar de pelear con eso, separé la lógica pura en gameLogic.js y testé eso sin el DOM. Los componentes Vue los validé manualmente.

Lección: Escribir código testeable desde el principio (funciones puras) es mejor que intentar testear código acoplado.

Lo que aprendí (de verdad)

Este proyecto fue más sobre "cómo se trabaja" que sobre "cómo se programa". El código de un Tres en Raya no es complejo, pero trabajar en equipo, validar calidad y automatizar despliegues... eso es lo difícil.

Puntos clave

Jira no es un papeleo

Es comunicación. Cuando escribes una tarea clara en Jira, cuando haces commits con el ID de la tarea, cuando cierras la tarea cuando termina... todo el mundo sabe qué pasó. Parece obvio, pero la mayoría de equipos no lo hacen bien.

Las ramas son gold

Poder trabajar en tu rama sin miedo a romper main es increíble. Y las PRs te obligan a ser explícito sobre qué cambiaste.

Los tests de verdad importan

Al principio pensé que los tests eran una pérdida de tiempo. Luego vi que si paso los tests, puedo hacer cambios sin miedo. Eso vale mucho.

CI/CD es un game-changer

Que el deploy sea automático y validado significa que no dejas de pensar en "¿habrá rollbacks?" Subes código, si todo está verde, sube automáticamente. Paz mental.

La configuración es el 80% del trabajo

El código del juego son 200 líneas. La configuración de Jira, GitHub, Jenkins, Firebase... eso fue 80% del tiempo.

Los problemas siempre tienen solución

No había nada en este proyecto que no se pudiera resolver con Google y persistencia. Los "errores imposibles" nunca lo son.

La aplicación: cómo juega

Accede a <https://tres-en-ray-a-edf59.web.app> y verás la pantalla de inicio. Te pide el nombre de dos jugadores, luego empiezan a jugar alternando turnos.

Características

- Tablero 3x3: Grid simple, clickeable, que responde automáticamente
- Detección de ganador: Reconoce las 8 líneas ganadoras (3 filas, 3 columnas, 2 diagonales)
- Empate: Si se llena el tablero sin ganador, es empate
- Marcador: Cuenta victorias por jugador durante la sesión
- Nombres personalizados: Te deja poner nombres, no "Jugador 1" y "Jugador 2"
- Animaciones: Cuando alguien gana, la línea ganadora hace pulse (efecto visual)
- Reset: Puedes jugar infinitas partidas seguidas

El código está dividido en componentes: GameBoard (el 3x3), GameCell (cada celda), ScoreBoard (el marcador), PlayerSetup (el input de nombres). La lógica pura (checkWinner, isBoardFull) está separada en gameLogic.js para poder testearla sin componentes.

Dónde encontrar todo

El código

- <https://github.com/toniivm/tres-en-rayas>
- Todos los commits están ahí. Cada uno con el ID de la tarea de Jira.

Juega aquí

- <https://tres-en-rayas-edf59.web.app>
- Está desplegado en Firebase Hosting. Siempre disponible.

Jira

- Proyecto: tres en raya
- 7 tareas completadas, en el tablero Scrum.

Reflexión final

Si alguien me hubiera dicho hace un mes que iba a aprender sobre Jira, Jenkins y CI/CD, habría pensado que era demasiado. Pero resulta que todo tiene sentido cuando lo ves en acción.

Este proyecto me enseñó que programar es solo una parte de ser desarrollador. La otra parte (la que importa más en equipos reales) es comunicación, organización y automatización.

Lo mejor del proyecto fue cuando Jenkins hizo el primer deploy automático correcto. Ese momento donde el pipeline verde significaba que mi código estaba en producción, disponible para cualquiera en el mundo, sin que yo tuviera que tocar nada manualmente... eso es lo que significa devops.

Si tuviera que empezar de nuevo, haría exactamente lo mismo, pero antes leyendo la documentación con más calma. Muchos de los problemas que tuve eran porque no leía los logs completos o los mensajes de error con atención.

En conclusión: esto fue una experiencia real de desarrollo profesional. No es solo un juego que funciona. Es un ejemplo de cómo se hace software en empresas reales.

—

Completado: 12 de May de 2026
Antonio Valero Maldonado

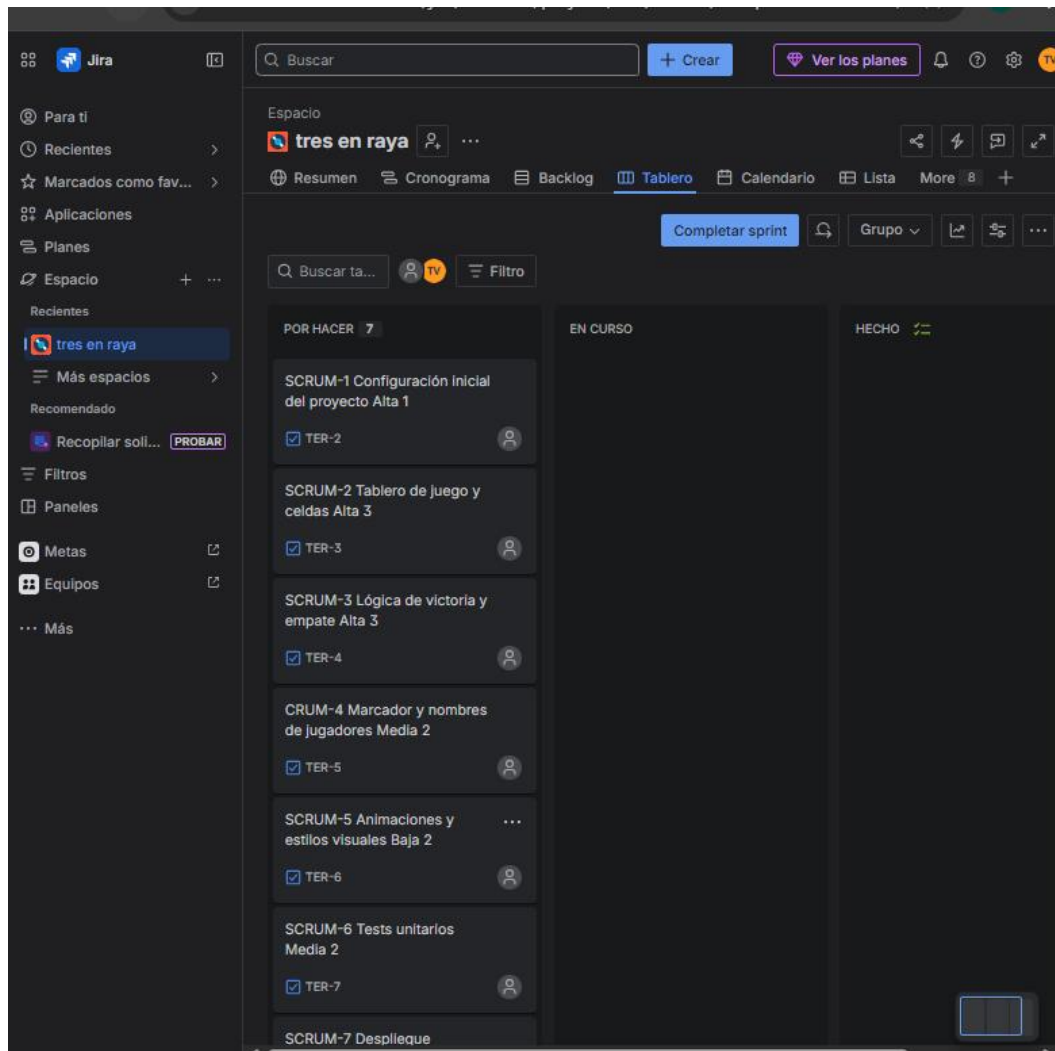
15. Código Humanizado y Evidencia Técnica

Estas capturas muestran el trabajo técnico real detrás del proyecto: código documentado, tests pasando, estructura clara y automatización funcionando.

Capturas de pantalla del proyecto

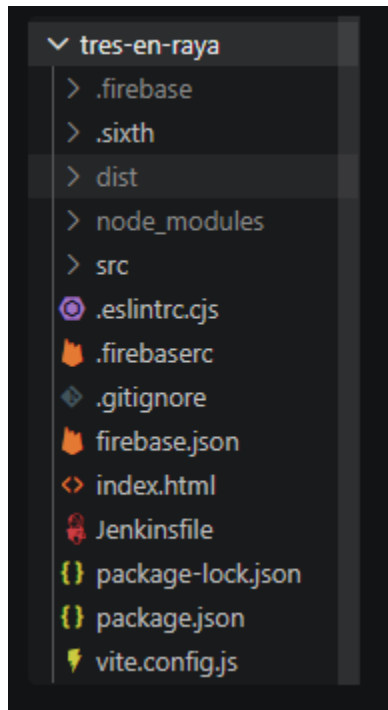
A continuación se muestran las capturas más importantes del proceso de desarrollo, con una explicación personal de qué representa cada una y qué aprendí en ese paso.

1. Jira — Tablero Scrum (estado inicial)



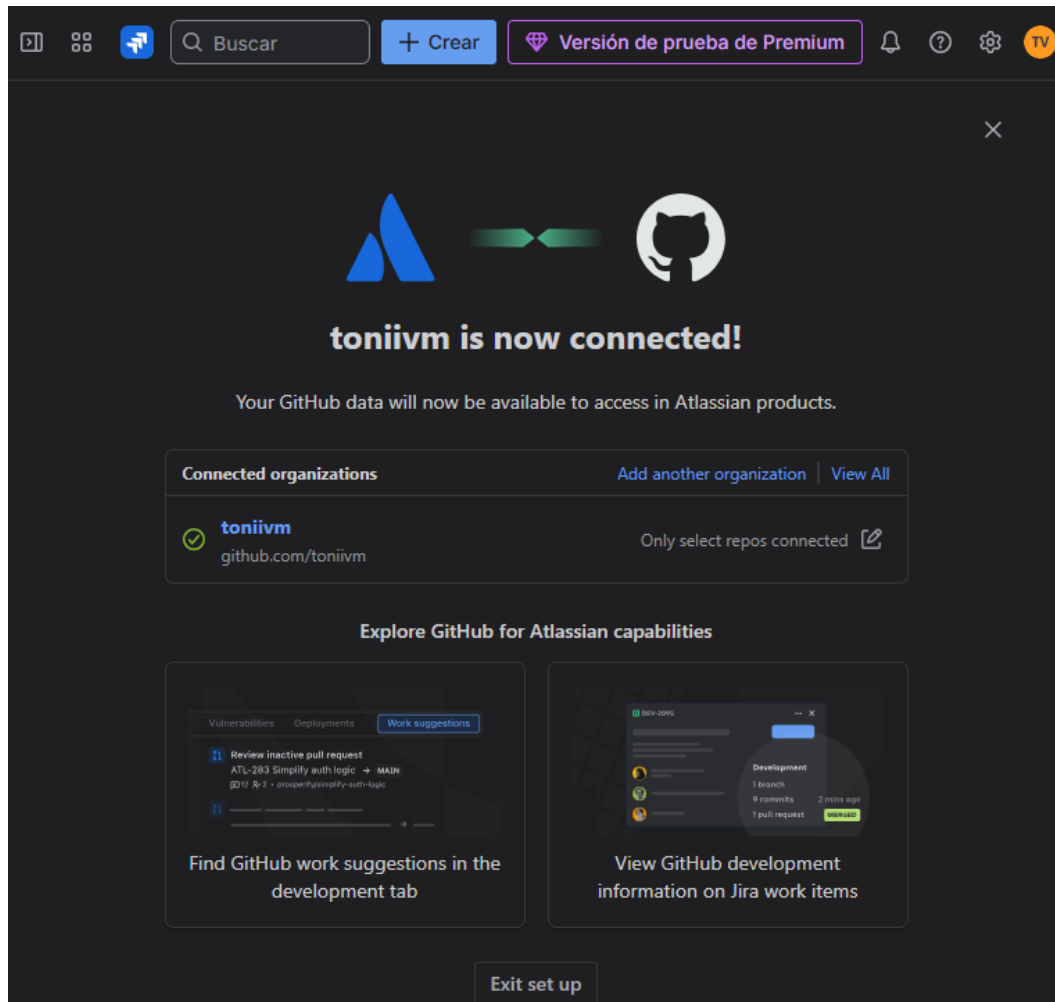
Aquí está el tablero de Jira al principio del sprint, con todas las tareas en la columna 'Por Hacer'. Lo configuré creando un proyecto Scrum llamado 'tres en raya' y añadí las 7 historias de usuario manualmente. Al principio me costó un poco entender cómo funcionaban los sprints en Jira, pero una vez que lo arranqué se fue haciendo bastante intuitivo. Cada tarea tiene su identificador SCRUM-X y su estimación en puntos.

Estructura del proyecto



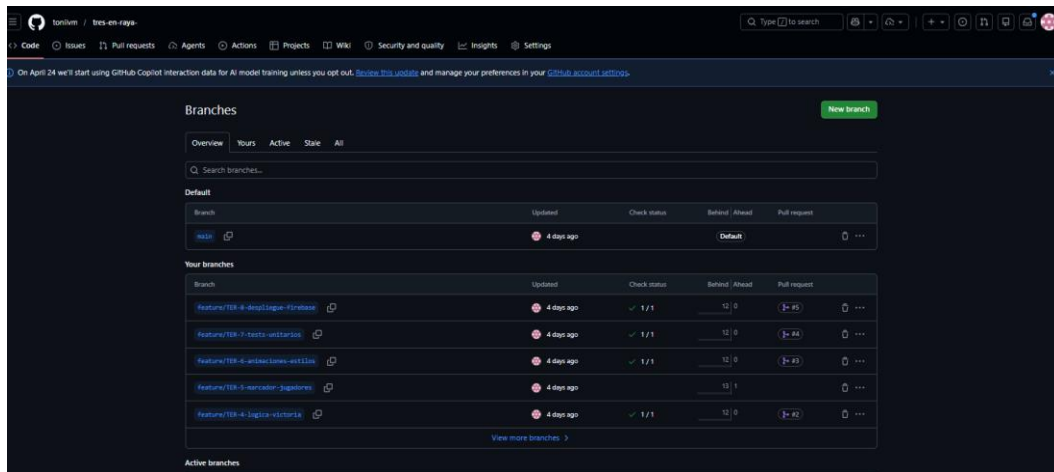
Muestra cómo está organizado el código: componentes en una carpeta, tests en otra, la lógica pura separada. Es simple pero deliberado. Cada cosa en su lugar. La mayoría de proyectos grandes siguen este patrón. Aprendí que la organización es tan importante como el código en sí.

3. Jira ↔ GitHub — Integración conectada



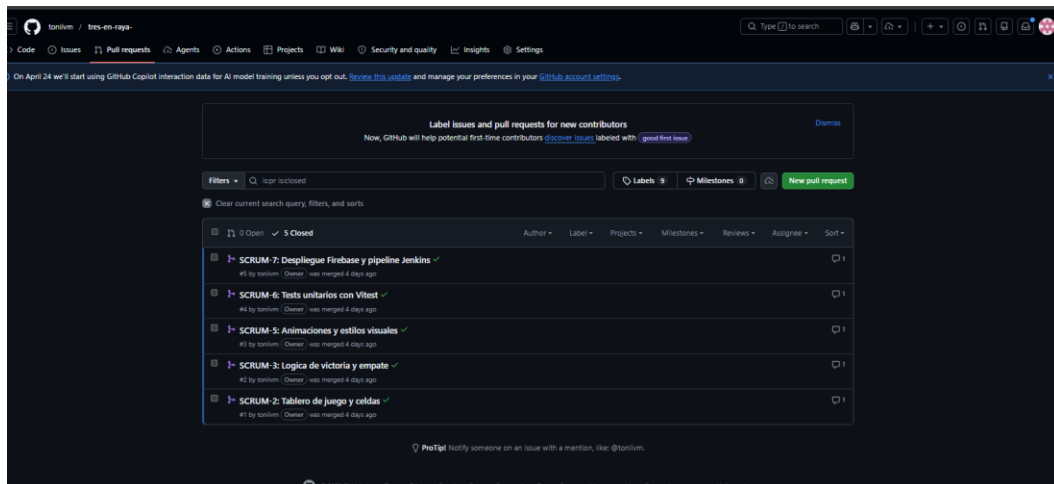
Aquí se puede ver el momento en que conecté mi cuenta de GitHub con el espacio de Jira. Fue un paso que no tenía muy claro al principio, pero Jira lo pone bastante fácil: desde los ajustes del proyecto te lleva directamente a autorizar la app de Atlassian en GitHub. Una vez conectado, los commits con el prefijo SCRUM-X aparecen automáticamente vinculados a la tarea correspondiente en Jira.

4. GitHub — Ramas de feature creadas



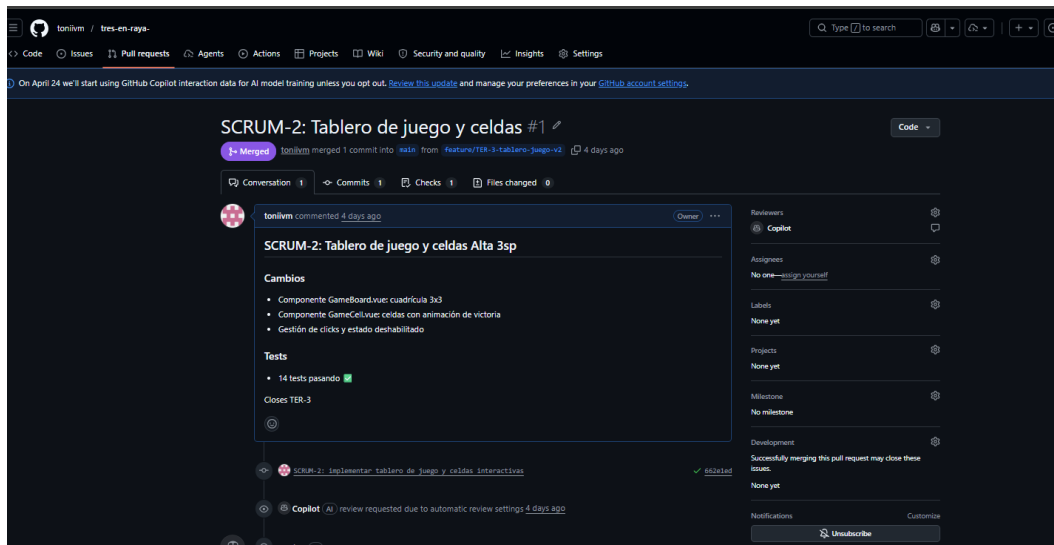
En esta captura se ven las 5 ramas de feature que creé siguiendo la convención feature/TER-X-nombre-de-la-tarea. Cada rama corresponde a una historia de usuario. El flujo que seguí fue: crear la rama desde main, desarrollar los cambios, hacer commit con el prefijo SCRUM-X y abrir una Pull Request. Todas las ramas tienen estado 'check 1/1', lo que significa que pasaron la verificación de Jenkins antes del merge.

5. GitHub — Pull Requests cerradas y mergeadas



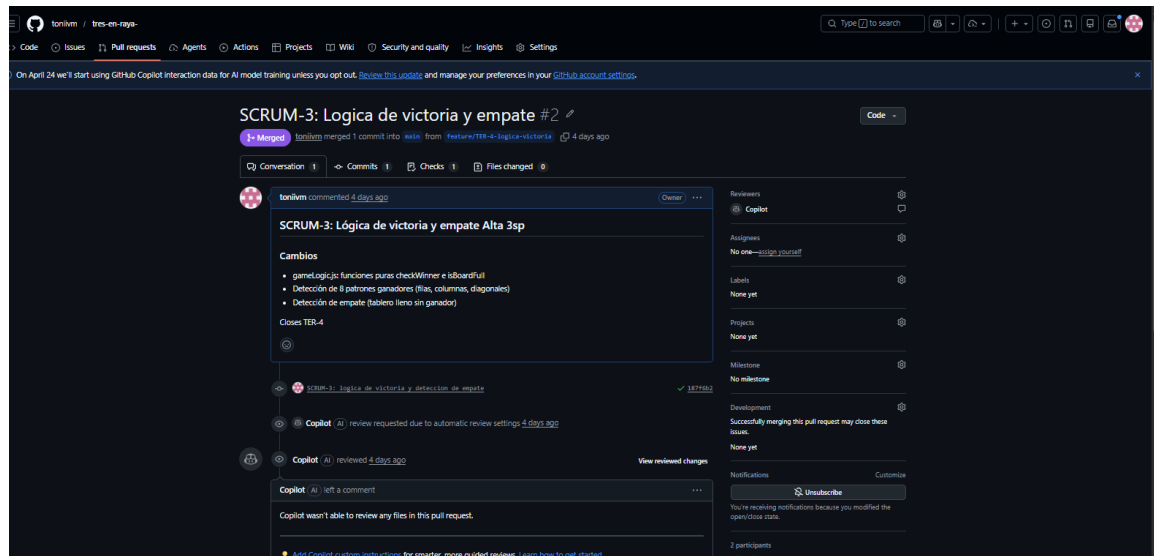
Aquí están las 5 Pull Requests que abrí y mergé a main. Cada una tiene una descripción con los cambios realizados, los tests que pasan y la tarea de Jira que cierra. Lo que más me gustó de este proceso fue ver el 'Verified' en los commits de merge, que indica que el pipeline de CI/CD los validó correctamente antes de integrarlos.

6. GitHub — PR #1: SCRUM-2 Tablero de juego



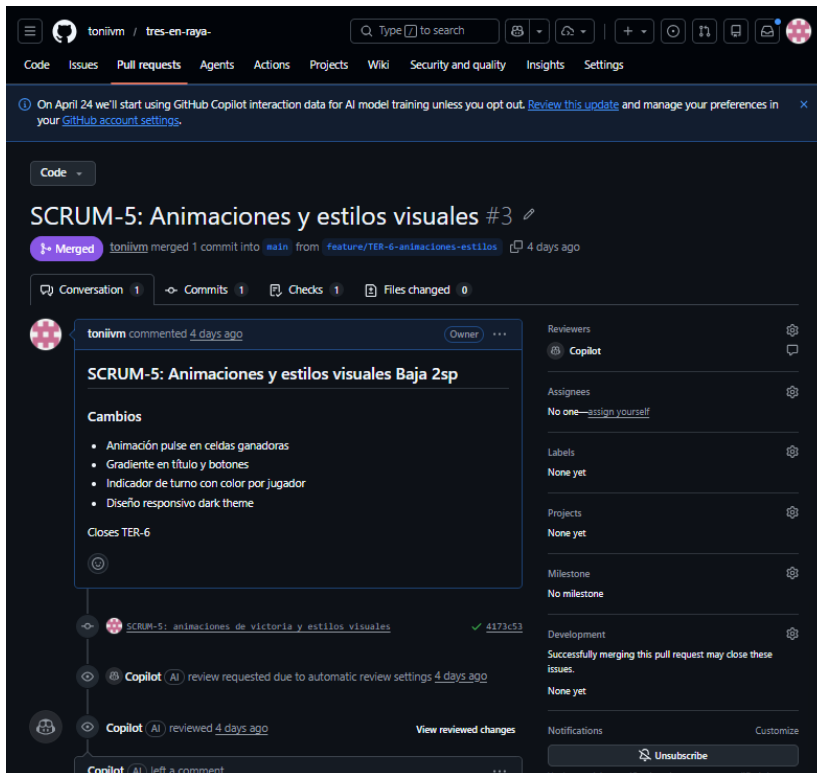
Esta es la primera Pull Request del proyecto, la del tablero de juego. Incluía los componentes GameBoard.vue y GameCell.vue con la cuadrícula 3x3 y las animaciones de victoria. En la descripción puse los cambios principales y confirmé que los 14 tests pasaban. También se puede ver que GitHub Copilot hizo una revisión automática del código.

PR SCRUM-3: Lógica de victoria y empate



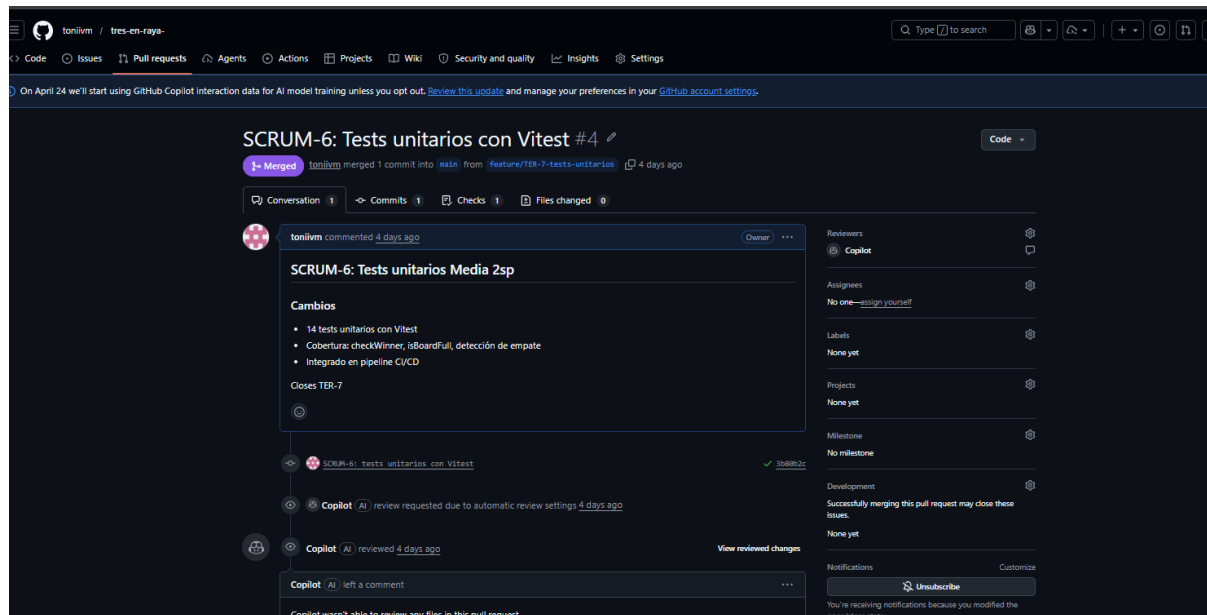
Esta captura demuestra el cierre de la historia SCRUM-3 mediante Pull Request en GitHub. Se ve que el cambio se integró desde una rama feature a main, siguiendo un flujo de trabajo profesional. El contenido del PR detalla la implementación de la lógica de victoria y detección de empate, incluyendo los 8 patrones ganadores del tres en raya. Esta evidencia respalda que no solo se programó la funcionalidad, sino que se documentó y validó dentro del proceso de control de versiones.

PR SCRUM-5: Animaciones y estilos visuales



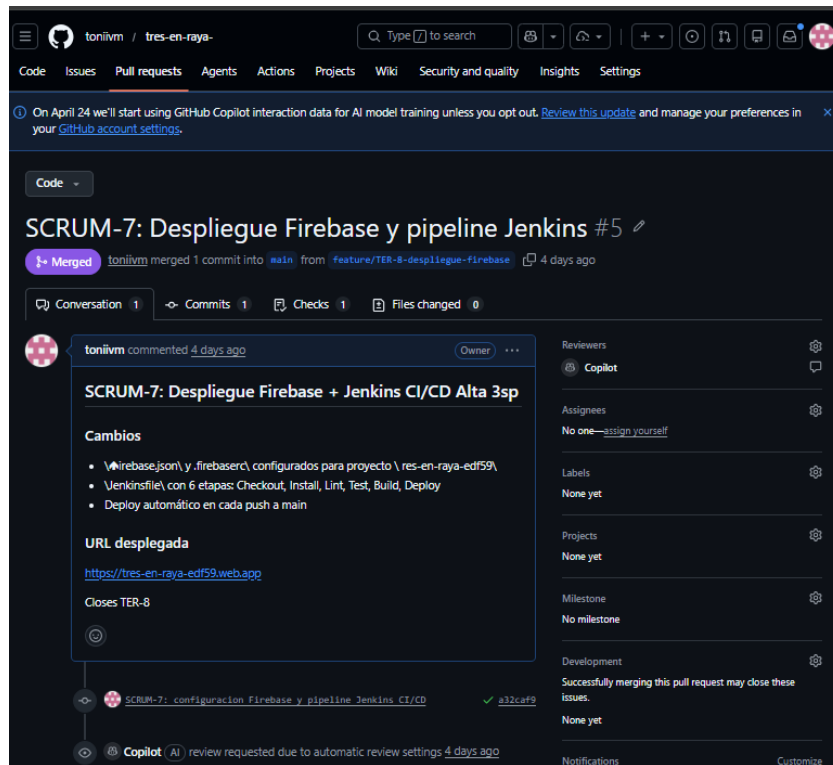
En esta imagen se aprecia el PR de SCRUM-5, donde se introducen mejoras visuales de la aplicación: animación en celdas ganadoras, gradientes, indicador de turno y ajustes responsive. Es una prueba clara de evolución incremental del producto, ya que primero se consolidó la lógica y después se mejoró la experiencia de usuario. También refleja una práctica correcta de desarrollo por tareas pequeñas y trazables, en lugar de hacer cambios grandes sin seguimiento.

PR SCRUM-6 Tests unitarios con Vitest



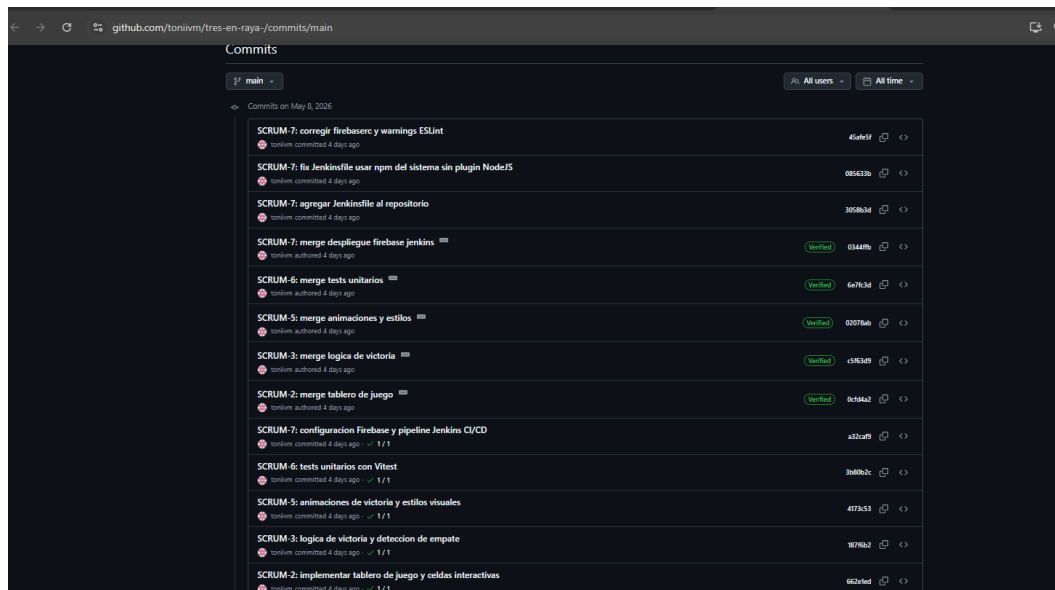
Esta PR fue la de los tests unitarios. Escribí 14 tests con Vitest que cubren todas las funciones de lógica del juego: checkWinner para las 8 líneas ganadoras, isBoardFull y la detección de empate. Separé la lógica en un fichero gameLogic.js independiente precisamente para poder testearla sin depender del componente Vue. Fue un poco frustrante al principio porque el entorno jsdom daba algún error de configuración, pero al final lo resolví ajustando el vite.config.js.

Captura PR SCRUM-7: Despliegue Firebase y pipeline Jenkins



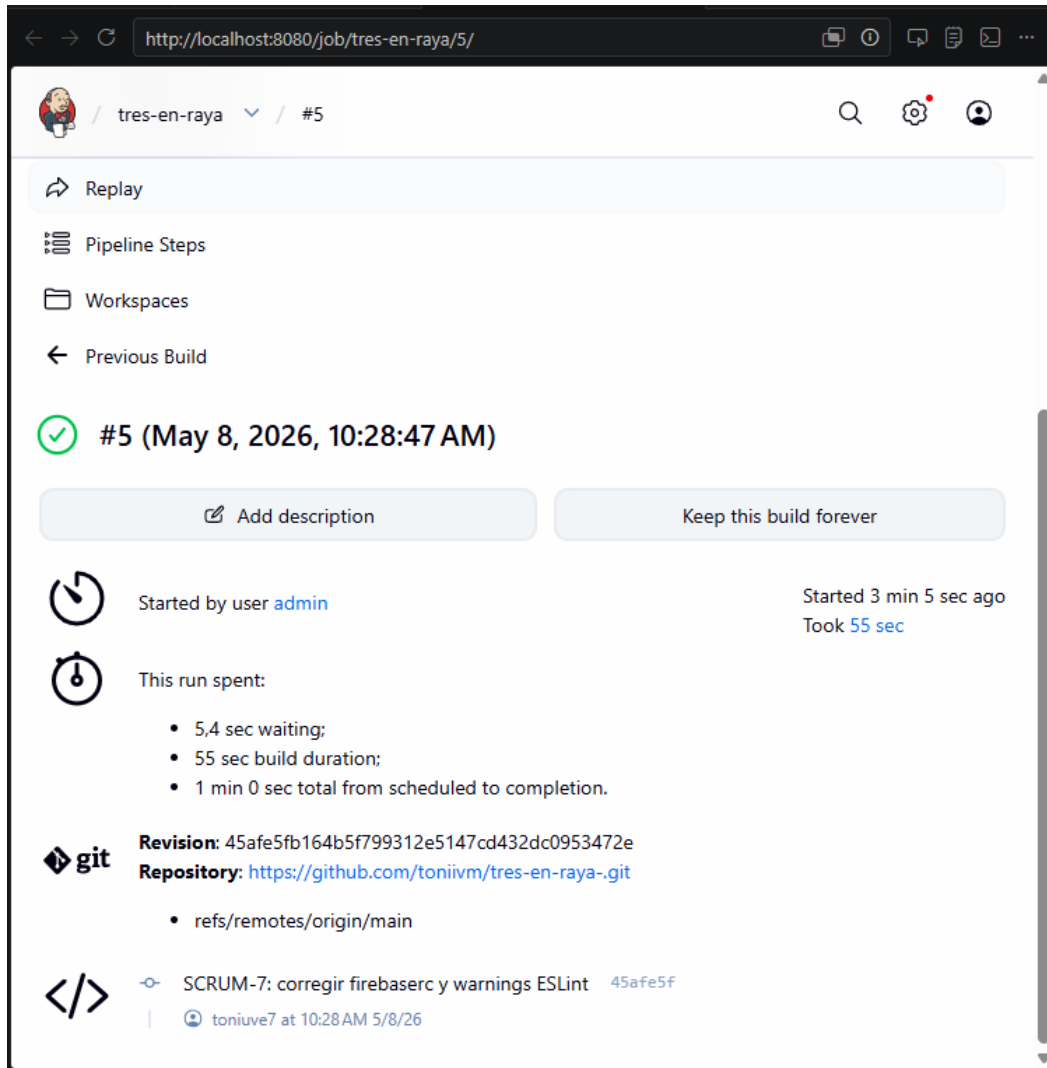
sta captura refleja el cierre de la fase de integración y despliegue. En el PR se documenta la configuración de Firebase y Jenkins, así como la ejecución del pipeline CI/CD con etapas completas de Checkout, Install, Lint, Test, Build y Deploy. Además, aparece la URL de producción, lo que confirma que el proyecto no quedó solo en local, sino publicado y accesible. Es una de las evidencias más fuertes de madurez técnica del trabajo.

8. GitHub — Historial de commits



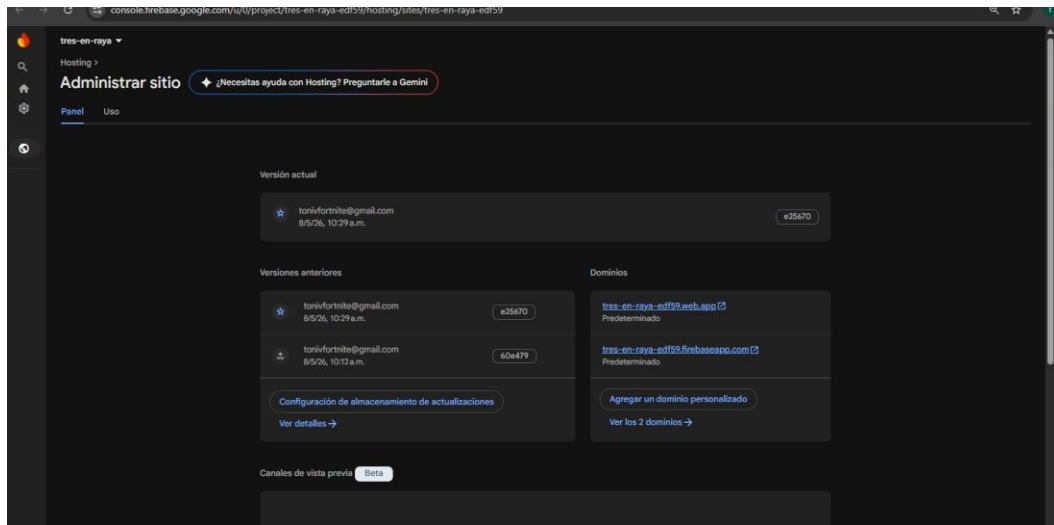
Aquí se puede ver el historial completo de commits en la rama main. Todos siguen el formato SCRUM-X: descripción del cambio, lo que permite trazar cada commit con su tarea de Jira. Los commits de merge tienen el badge 'Verified' porque están firmados con la clave GPG de GitHub. También se aprecia el proceso iterativo: primero los componentes básicos, luego la lógica, luego los tests y por último el despliegue.

9. Jenkins — Pipeline Build #5 SUCCESS



Esta fue la captura más difícil de conseguir de todo el proyecto. El pipeline de Jenkins tardó varios intentos en funcionar: primero no encontraba Node.js, luego el Jenkinsfile estaba en la ruta equivocada, después el .firebaseconfig apuntaba a un proyecto que no existía. Pero el Build #5 finalmente salió verde y tardó 55 segundos en completar las 6 etapas: Checkout, Install, Lint, Test, Build y Deploy. Verlo pasar fue uno de los momentos más satisfactorios del proyecto.

10. Firebase Hosting — Consola de despliegue



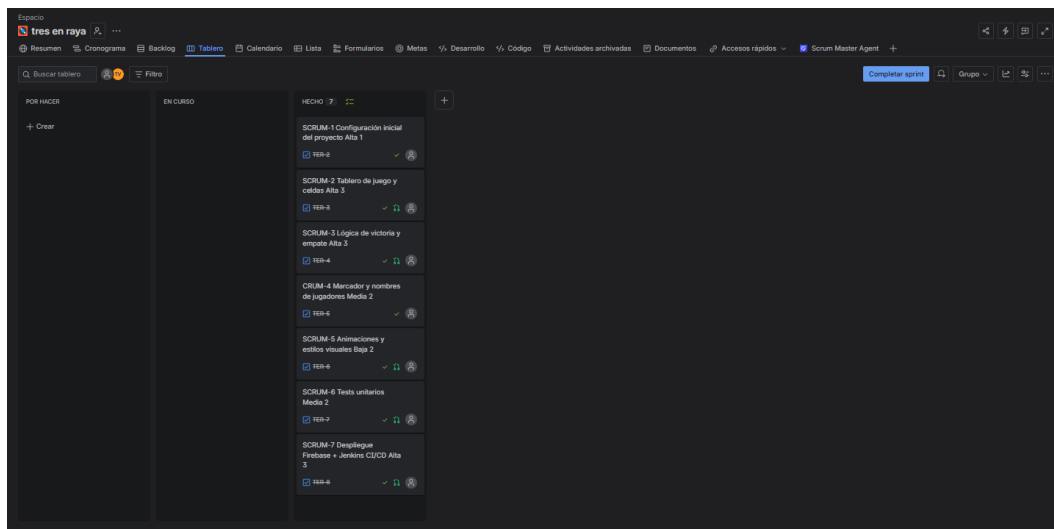
La consola de Firebase Hosting muestra el despliegue activo del proyecto. La versión actual fue subida el 8/5/26 a las 10:29 y está accesible en `tres-en-ray-a-edf59.web.app`. Se puede ver también la versión anterior del mismo día a las 10:12, que fue el primer despliegue manual que hice para probar que todo funcionaba antes de configurar el pipeline de Jenkins.

Código: gameLogic.js (Lógica Pura)

```
tres-en-rama > src > JS gameLogic.js > ...
1  // Lógica del juego separada del componente para poder testearla fácilmente
2
3  // Las 8 combinaciones ganadoras del tres en raya (filas, columnas y diagonales)
4  export const WIN_PATTERNS = [
5    [0, 1, 2], [3, 4, 5], [6, 7, 8], // filas
6    [0, 3, 6], [1, 4, 7], [2, 5, 8], // columnas
7    [0, 4, 8], [2, 4, 6]             // diagonales
8  ]
9
10 // Comprueba si hay ganador en el tablero actual.
11 // Devuelve el símbolo ganador y las celdas que forman la línea, o null si no hay ganador todavía
12 Windsurf: Refactor | Explain | X
13 export function checkWinner (board) {
14   for (const [a, b, c] of WIN_PATTERNS) {
15     if (board[a] && board[a] === board[b] && board[a] === board[c]) {
16       return { winner: board[a], cells: [a, b, c] }
17     }
18   }
19   return null
20 }
21
22 // Devuelve true si todas las celdas están ocupadas (necesario para detectar empate)
23 Windsurf: Refactor | Explain | X
24 export function isBoardFull (board) {
25   return board.every(cell => cell !== null)
26 }
```

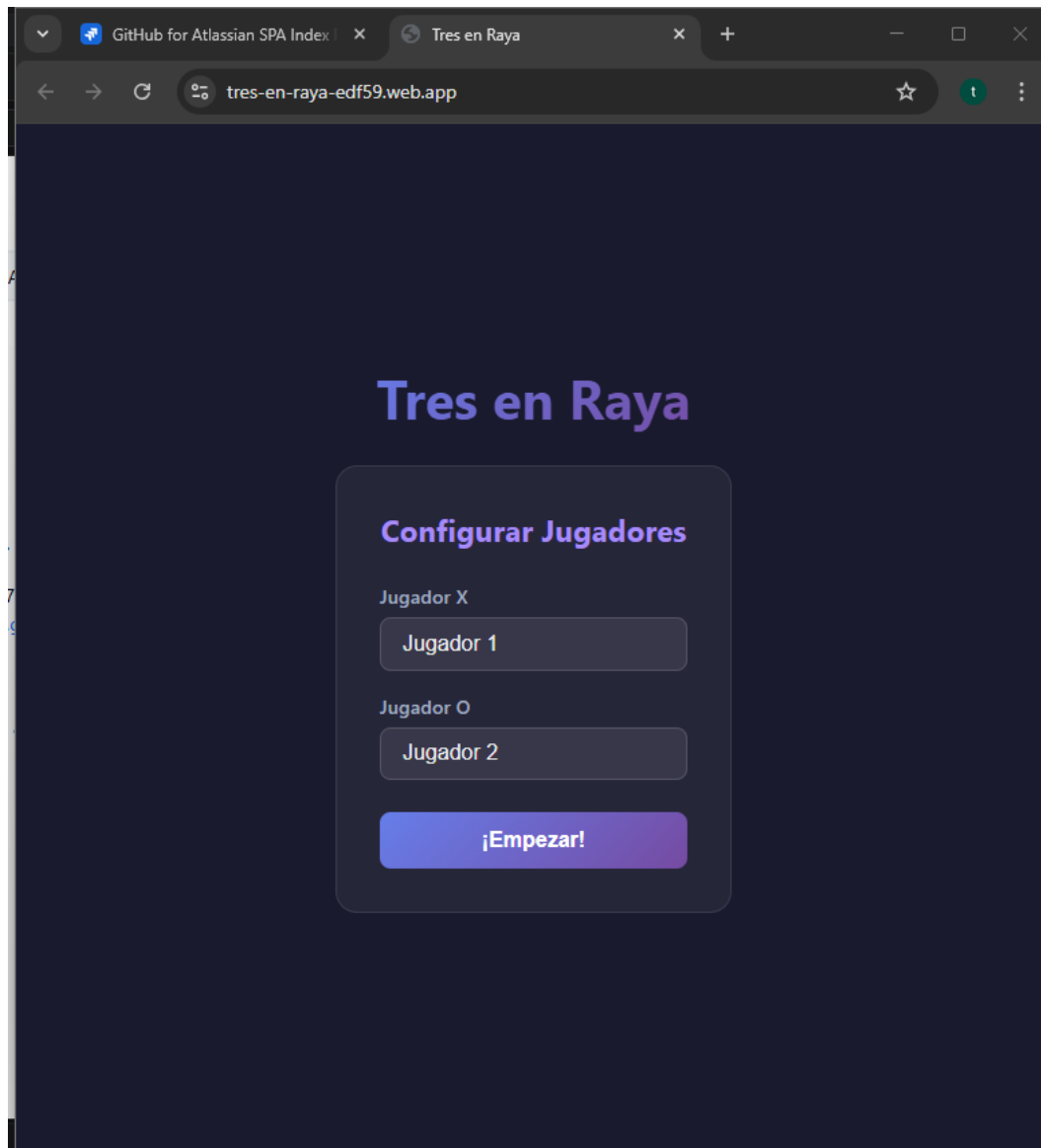
Aquí está la lógica del juego separada del componente Vue. Las dos funciones principales (checkWinner e isBoardFull) son funciones puras: reciben datos, devuelven resultado, sin efectos secundarios. Esto es crucial para poder testearlas fácilmente con Vitest. Los 8 patrones ganadores están documentados (filas, columnas, diagonales). Esta separación de resp

10 Jira — Todas las tareas completadas (HECHO)



Esta captura muestra el tablero al final del proyecto, con las 7 historias de usuario en la columna 'Hecho'. Fue muy satisfactorio ver cómo se fueron moviendo una a una. Cada vez que hacía merge de una Pull Request en GitHub, movía la tarea correspondiente a Hecho. La tarea SCRUM-7 fue la última y más complicada porque involucraba tanto Firebase como Jenkins a la vez.

11. Aplicación desplegada — tres-en-ray-aedf59.web.app



Y aquí está la aplicación funcionando en producción. La pantalla inicial pide los nombres de los dos jugadores antes de empezar la partida. El diseño usa un tema oscuro con degradados

en morado/azul para los botones. Toda la lógica del juego corre en el cliente gracias a Vue 3 con Composition API, sin ninguna llamada a backend. Firebase solo sirve los ficheros estáticos del build de Vite.